Real Time Air Drawing Application Using OpenCV And Machine Learning

A Project Phase-II Report Submitted

In partial fulfillment of the requirement for the award of the degree of

Bachelor of Technology *in* Artificial Intelligence and Machine Learning

by

P. DILEEP KUMAR	**22N31A66D8**
P. SAI CHARAN	**22N31A66E7**
R. SURYA PRAKASH REDDY	**22N31A66F4**

Under the esteemed Guidance of

P. SWAPNA
Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)
MALLA REDDY COLLEGE OF ENGINEERING AND TECHNOLOGY
(Autonomous Institution - UGC, Govt. of India)
(Affiliated to JNTU, Hyderabad, Approved by AICTE, Accredited by NBA & NAAC – 'A' Grade, ISO 9001:2015 Certified) Maisammaguda (v), Near Dullapally, Via: Kompally, Hyderabad – 500 100, Telangana State, India.
website: www.mrcet.ac.in

2025-2026

DECLARATION

We hereby declare that the project entitled **"Real-Time Air Drawing Application Using OpenCV and Machine Learning"** submitted to **Malla Reddy College of Engineering and Technology,** affiliated t**o** Jawaharlal Nehru Technological University Hyderabad (JNTUH) for the award of the degree of **Bachelor of Technology** in **Artificial Intelligence and Machine Learning** is a result of original research work done by us.

It is further declared that the project report or any part thereof has not been previously submitted to any University or Institute for the award of degree or diploma.

P. Dileep Kumar (22N31A66D8)
P. Sai Charan (22N31A66E7)
R. Surya Prakash Reddy (22N31A66F4)

MALLA REDDY COLLEGE
OF ENGINEERING & TECHNOLOGY
(AUTONOMOUS INSTITUTION - UGC, GOVT. OF INDIA)
Affiliated to JNTUH; Approved by AICTE, NBA-Tier 1 & NAAC with A-GRADE | ISO 9001:2015

CERTIFICATE

This is to certify that this is the bonafide record of the project titled "**Real-Time Air Drawing Application Using OpenCV and Machine Learning**" submitted by **P. Dileep Kumar (22N31A66D8)**, **P. Sai Charan (22N31A66E7)**, **R. Surya Prakash Reddy (22N31A66F4)** of B.Tech in the partial fulfillment of the requirements for the degree of **Bachelor of Technology** in **Artificial Intelligence and Machine Learning**, Dept. of CSE(AI&ML) during the year 2025-2026. The results embodied in this project report have not been submitted to any other university or institute for the award of any degree or diploma.

P. Swapna
Assistant Professor
INTERNAL GUIDE

Dr. A. Nagaraju
Professor
HEAD OF THE DEPARTMENT

EXTERNAL EXAMINER

Date of Viva-Voce Examination held on: _____

ACKNOWLEDGEMENT

The Project Phase II work carried out by our team in the Department of CSE (Artificial Intelligence and Machine Learning), Malla Reddy College of Engineering and Technology, Hyderabad. ***This work is original and has not been submitted in part or full for any degree or diploma of any other university.***

We wish to acknowledge our sincere thanks to our project guide **P. Swapna**, Assistant Professor for formulation of the problem, analysis, guidance and her continuous supervision during the course of work.

We acknowledge our sincere thanks to **Dr. VAKA MURALI MOHAN**, Principal, **Dr. A NAGARAJU**, Head of the Department and **Mr. T. Shiva Kumar**, Coordinator, faculty members of CSE(AI&ML) Department for their kind cooperation in making this Project Phase II work a success.

We extend our gratitude to **Sri. Ch. Malla Reddy**, Founder Chairman MRGI and **Sri. Ch. Mahender Reddy**, Secretary MRGI, **Dr. Ch. Bhadra Reddy**, President MRGI, **Sri. Ch. Shalini Reddy**, Director MRGI, **Sri. P. Praveen Reddy**, Director MRGI, for their kind cooperation in providing the infrastructure for completion of our Project Phase II.

We acknowledge our special thanks to the entire teaching faculty and non-teaching staff members of the CSE (Artificial Intelligence and Machine Learning) Department for their support in making this project work a success.

PALLELAPATI DILEEP KUMAR- 22N31A66D8
POTHAGANI SAI CHARAN- 22N31A66E7
RAVULA SURYA PRAKASH REDDY-22N31A66F4

ABSTRACT

Real-Time Air Drawing application that enables users to draw on a virtual canvas using hand gestures captured through a webcam. The system leverages computer vision techniques provided by **OpenCV** and hand tracking capabilities from Media Pipe to detect and interpret hand movements without requiring any physical contact or additional hardware. The application tracks the user's hand in real time and identifies key landmarks, particularly the tip of the index finger and thumb. By analyzing the relative positions of these landmarks, the system distinguishes between drawing and non-drawing gestures. When the index finger is extended and separated from the thumb, drawing is enabled, allowing users to sketch freely in the air. Conversely, bringing the thumb close to the index finger temporarily disables drawing, preventing unintended strokes.

A virtual interface is overlaid on the screen, providing options to select different colors(blue, green, red, and yellow) and a clear function to reset the canvas. The drawing data is stored using dynamic data structures to ensure smooth and continuous stroke rendering. The system simultaneously displays the live video feed with drawn strokes and a separate digital canvas showing the final artwork. This approach demonstrates an intuitive human-computer interaction technique, combining machine learning-based hand tracking with real-time image processing. The application can be extended for use in virtual whiteboards, touchless interfaces, and interactive educational tools, especially in environments where physical contact with devices is limited.

TABLE OF CONTENTS

CONTENTS	Page No.
1. INTRODUCTION	**1**
1.1. Problem Statements	2
1.2. Objective	2-3
2. LITERATURE SURVEY	**4**
2.1. Existing System	4
2.2. Proposed System	4-5
3. SYSTEM REQUIREMENTS	**6**
3.1. Software and Hardware Requirement	6
3.2. Functional and Non-Functional Requirements	7-8
3.3. Other Requirements	8-9
4. SYSTEM DESIGN	**10**
4.1. Architecture Diagram	10
4.2. UML Diagrams	11-14
5. IMPLEMENTATION	**15**
5.1. Algorithms	15-16
5.2. Architectural Components	17-18
5.3. Feature Extraction	18-20
5.4. Packages/Libraries Used	20-21
5.5 Source Code	22-26
5.6. Output Screens	27-30
6. SYSTEM TESTING	**31**
6.1. Test Cases	31-32
6.2. Results and Discussions	32-33
6.2.1. Datasets	34
6.3. Performance Evaluation	34-35
7. CONCLUSION & FUTURE ENHANCEMENTS	**36-37**
8. REFERENCES	**38**

LIST OF FIGURES

Fig. No	Figure Title	Page no.
4.1	Architecture Diagram	10
4.2	Use Case Diagram	11
4.3	Class Diagram	12
4.4	Sequential Diagram	13
4.5	Activity Diagram	14
5.6	Output Screens	27-30

LIST OF TABLES

S. No	Table Name	Page No
3.1	Software Requirements	6
3.1	Hardware Requirements	6

LIST OF ABBREVIATIONS

S. No	ABBREVIATIONS
1.	CV – Computer Vision
2.	ML – Machine learning
3.	HCI – Human Computer Interaction
4.	RGB- Red Green Blue
5.	FPS – Frames Per second
6.	API – Application programming Interface

CHAPTER 1
INTRODUCTION

In recent years, the advancement of computer vision and machine learning technologies has significantly transformed the way humans interact with computers. Traditional input devices such as keyboards, mice, and touchscreens require physical contact, which may not always be convenient or suitable in all environments. With the increasing demand for touchless and intuitive interfaces, gesture-based interaction systems have gained considerable attention. The Real-Time Air Drawing Application is an innovative solution that allows users to draw virtually in the air using simple hand gestures captured through a webcam. This system utilizes computer vision techniques and machine learning-based hand tracking to interpret user movements and translate them into digital drawings in real time.

The proposed system leverages MediaPipe for accurate hand landmark detection and OpenCV for real-time image processing and rendering. By tracking specific hand landmarks such as the index finger and thumb, the system identifies gestures to enable or disable drawing. A virtual interface is created on the screen, allowing users to select colors, clear the canvas, and interact without any physical tools. This approach enhances human-computer interaction by providing a natural and engaging way to communicate with digital systems. The application has potential use cases in virtual whiteboards, interactive learning environments, creative design tools, and touchless control systems, making it highly relevant in modern technological and hygienic contexts.

1.1 Problem Statement

In today's digital world, most drawing and interaction systems rely on physical input devices such as a mouse, stylus, or touchscreen. These traditional methods limit user flexibility and are not suitable in scenarios where touchless interaction is required, such as maintaining hygiene, remote communication, or immersive virtual environments. Additionally, existing drawing tools do not provide a natural way of interaction that mimics real-world hand movements.

There is a need for an intelligent system that enables users to interact with computers in a more intuitive and contactless manner. Specifically, a system is required that can accurately detect and interpret hand gestures in real time and convert them into meaningful actions such as drawing, color selection, and canvas control. Such a solution would enhance user experience, improve accessibility, and open new possibilities in human-computer interaction using computer vision and machine learning techniques.

1.2 Objectives

1. **To develop a real-time air drawing system:**
   Design and implement a system that allows users to draw on a virtual canvas using hand gestures captured through a webcam.
2. **To implement hand tracking using machine learning:**
   Utilize MediaPipe to accurately detect and track hand landmarks for gesture recognition.
3. **To enable gesture-based drawing control:**
   Identify specific gestures (such as index finger movement and thumb proximity) to control drawing and non-drawing actions.
4. **To provide a virtual user interface:**
   Create an interactive interface with options such as color selection (blue, green, red, yellow) and canvas clearing.
5. **To ensure real-time performance:**
   Optimize the system to process video frames efficiently and provide smooth, lag-free drawing experience.

6. **To enhance human-computer interaction:**

   Develop a touchless and intuitive system that improves user engagement and usability.

CHAPTER 2

LITERATURE SURVEY

2.1 Existing System

Traditional drawing and interaction systems primarily depend on physical input devices such as a mouse, stylus, or touchscreen. These systems are widely used due to their simplicity and ease of implementation. However, they require direct physical contact, which limits their usability in touchless environments and reduces flexibility in user interaction. Moreover, such systems do not provide a natural way of interaction that mimics human gestures, making them less intuitive.

In recent years, some gesture-based systems have been developed using basic image processing techniques such as color detection and background subtraction. These systems typically track colored markers or gloves worn by users to detect motion. While these approaches are simple and cost-effective, they suffer from several limitations, including sensitivity to lighting conditions, background noise, and the need for additional accessories. As a result, their accuracy and robustness are often compromised, making them unsuitable for real-time and practical applications.

2.2 Proposed System

The proposed system introduces a real-time air drawing application that utilizes advanced computer vision and machine learning techniques to enable touchless interaction. It leverages MediaPipe, a robust framework for hand tracking, to detect and track hand landmarks with high accuracy. By identifying key points such as the index finger tip and thumb, the system interprets user gestures to control drawing operations.

The system uses OpenCV for real-time video processing and rendering. When the index finger is extended and separated from the thumb, the system enters drawing mode and records the finger's movement as strokes on a virtual canvas. When the thumb comes close to the index finger, drawing is temporarily disabled to avoid unintended lines.

Additionally, a virtual interface is provided on the screen, allowing users to select different colors and clear the canvas through simple gestures.

Unlike traditional systems, the proposed approach does not require any external devices such as gloves or markers, making it more user-friendly and efficient. It offers improved accuracy, robustness, and real-time performance. This system enhances human-computer interaction by providing a natural, intuitive, and contactless method of drawing, with potential applications in education, design, and interactive systems.

.

CHAPTER 3
SYSTEM REQUIREMENTS

3.1 Software and Hardware Requirement

- **Software Requirements**

Component	Specification
Operating System	Windows 10 or above
Programming Language	Python 3.10
Library	OpenCV,MediaPipe,NumPy
IDE	VS Code

- **Hardware Requirements**

Component	Specification
Processor	Intel Core i5 or above
RAM	Minimum 8GB
Storage	256GB or above
GPU	NVIDIA GeForce GTX 1650

3.2 Functional and Non-Functional Requirements

- **Functional Requirements**
  1. **Video Input Module**
     - **Functionality:** Captures live video stream from the webcam
     - **Requirements:** Continuous frame capture with minimal delay.
  2. **Hand Detection Module**
     - **Functionality:** Detects and tracks hand landmarks using MediaPipe.
     - **Requirements:** Accurate detection of fingers in real time.
  3. **Gesture Recognition Module**
     - **Functionality:** Identifies gestures such as drawing and stopping using finger positions.
     - **Requirements:** Reliable interpretation of index finger and thumb positions.
  4. **Drawing Module**
     - **Functionality:** Draws lines on the virtual canvas based on finger movement.
     - **Requirements:** Smooth and continuous stroke rendering.
  5. **User Interface Module**
     - **Functionality:** Provides options like color selection and clear canvas.
     - **Requirements:** Easy interaction using hand gestures.
  6. **Display Module**

     Functionality: Shows live video feed and drawing canvas simultaneously.

     Requirements: Clear and real-time visualization.

- **Non-Functional Requirements**
  1. **Performance**
     - The system must operate in real time with minimal latency.
     - It should process frames efficiently without lag.

2. **Reliability**
   - The system should consistently detect hand gestures under normal conditions.
   - It should handle temporary detection failures gracefully.
3. **Usability**
   - The interface should be simple and intuitive.
   - Users should be able to interact without prior training.
4. **Scalability**
   - The system should support future enhancements such as multi-hand tracking or additional gestures.
5. **Maintainability**
   - The code should be modular and easy to update or debug.
6. **Security**
   - The system should ensure safe handling of camera input without storing sensitive data..

- **Other Requirements**

1. Data Requirements

These focus on the kind and quality of data the system relies on:

- Input Data: Real-time video frames captured from webcam
- Data Size: RGB images.
- Requirement: Continuous and clear video feed for accurate detection

2. Integration Requirements

Focus on the protection of data and system:

- Integration of MediaPipe for hand tracking.
- Integration of OpenCV for video processing and drawing.
- Smooth interaction between modules

3. System Requirements

How your system connects with others:

- Smooth interaction between modules
- Proper camera positioning.
- Minimum system resources to ensure real-time execution.

4. AI/ML Model Requirements

- Use of pre-trained MediaPipe hand tracking model.
- High accuracy in landmark detection.
- Fast inference for real-time interaction.

CHAPTER 4

SYSTEM DESIGN

4.1 Architecture Diagram

An architecture diagram is a visual representation of a system's structure, showcasing how its components are interconnected, how they communicate, and how data flows through the system, serving as a blueprint for understanding and communicating complex designs.

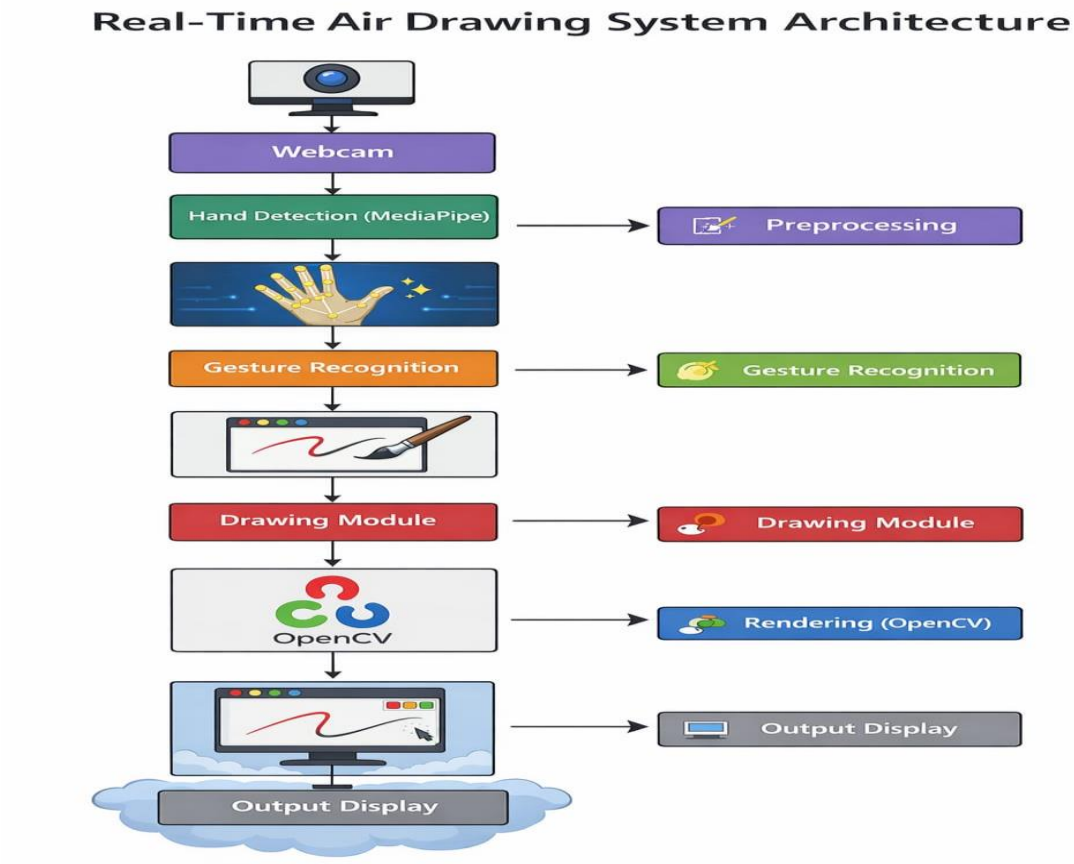


Fig 4.1 System Architecture

4.2 UML Diagrams

4.2.1 Use case diagram

Use Case during requirement elicitation and analysis to represent the functionality of the system. Use case describes a function by the system that yields a visible result for an actor. The identification of actors and use cases result in the definitions of the boundary of the system i.e., differentiating the tasks accomplished by the system and the tasks accomplished by its environment.

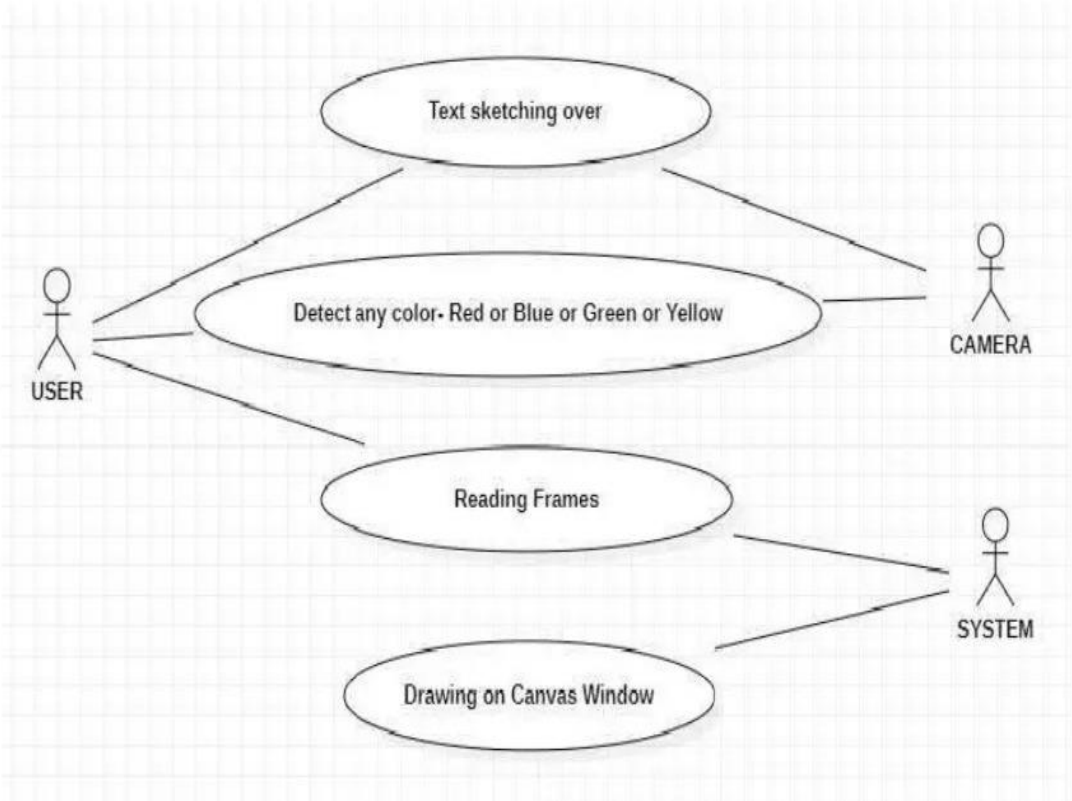


Fig 4.2 Use Case Diagram

4.2.2 Class Diagram

Class diagrams model class structure and contents using design elements such as classes, packages and objects. Class diagram describes the different perspective when designing a system-conceptual, specification and implementation. Classes are composed of three things: name, attributes, and operations. Class diagram also displays relationships such as containment, inheritance, association etc. The association relationship is most common relationship in a class diagram. The association shows the relationship between instances of classes.

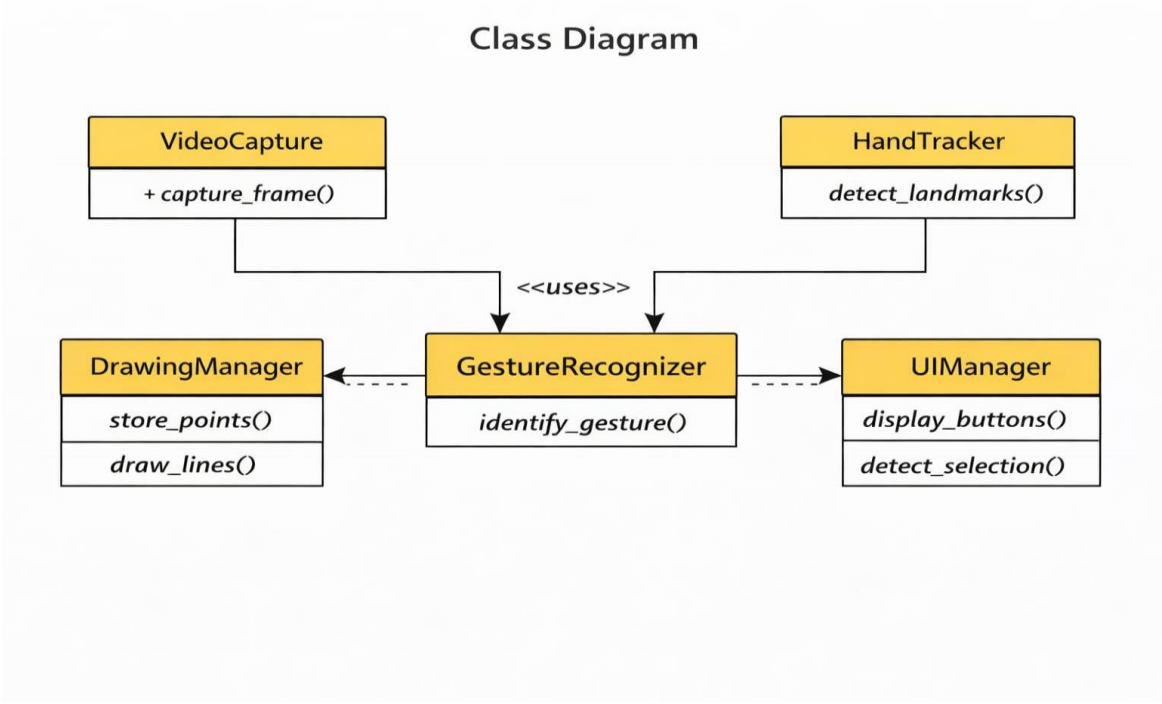


Fig 4.3 Class Diagram

4.2.3 Sequence Diagram

Sequence diagram displays the time sequence of the objects participating in the interaction. This consists of the vertical dimension (time) and horizontal dimension (different objects).

Objects: An object can be thought of as an entity that exists at a specified time and has a definite value, as well as a holder of identity. A sequence diagram depicts item interactions in chronological order. It illustrates the scenario's objects and classes, as well as the sequence of messages sent between them in order to carry out the scenario's functionality. In the Logical View of the system under development, sequence diagrams are often related with use case realizations. Event diagrams and event scenarios are other names for sequence diagrams. A sequence diagram depicts multiple processes or things that exist simultaneously as parallel vertical lines (lifelines), and the messages passed between them as horizontal arrows, in the order in which they occur. This enables for the graphical specification of simple runtime scenarios.

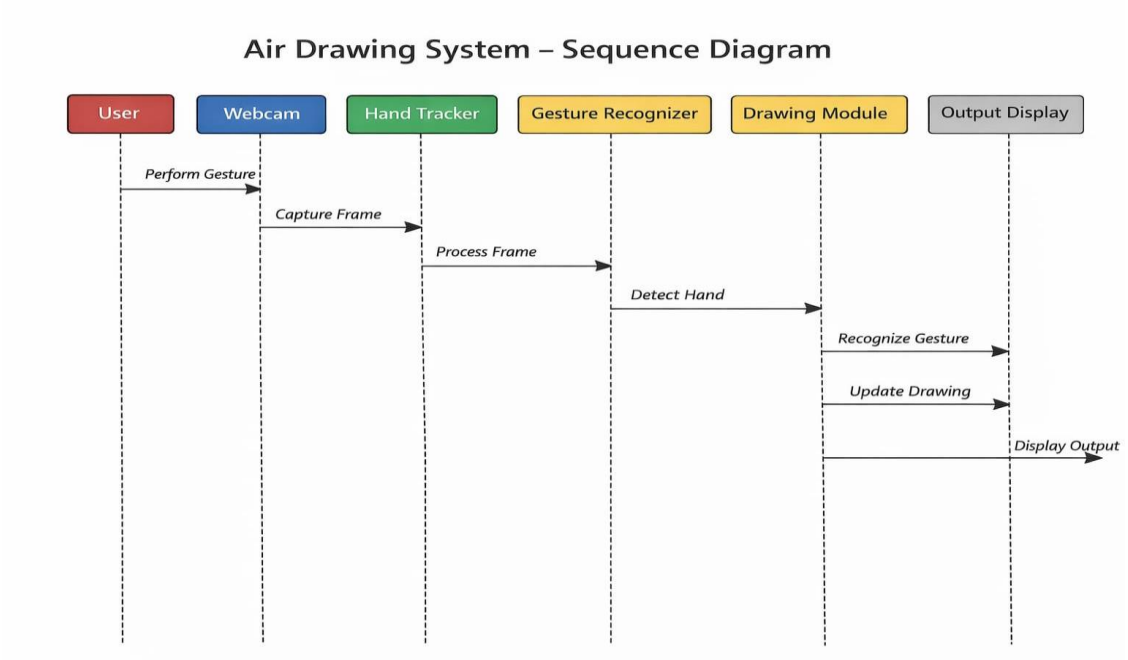


Fig 4.4 Sequence Diagram

4.2.4 Activity Diagram

The process flows in the system are captured in the activity diagram. Similar to a state diagram, an activity diagram also consists of activities, actions, transitions, initial and final states, and guard conditions.

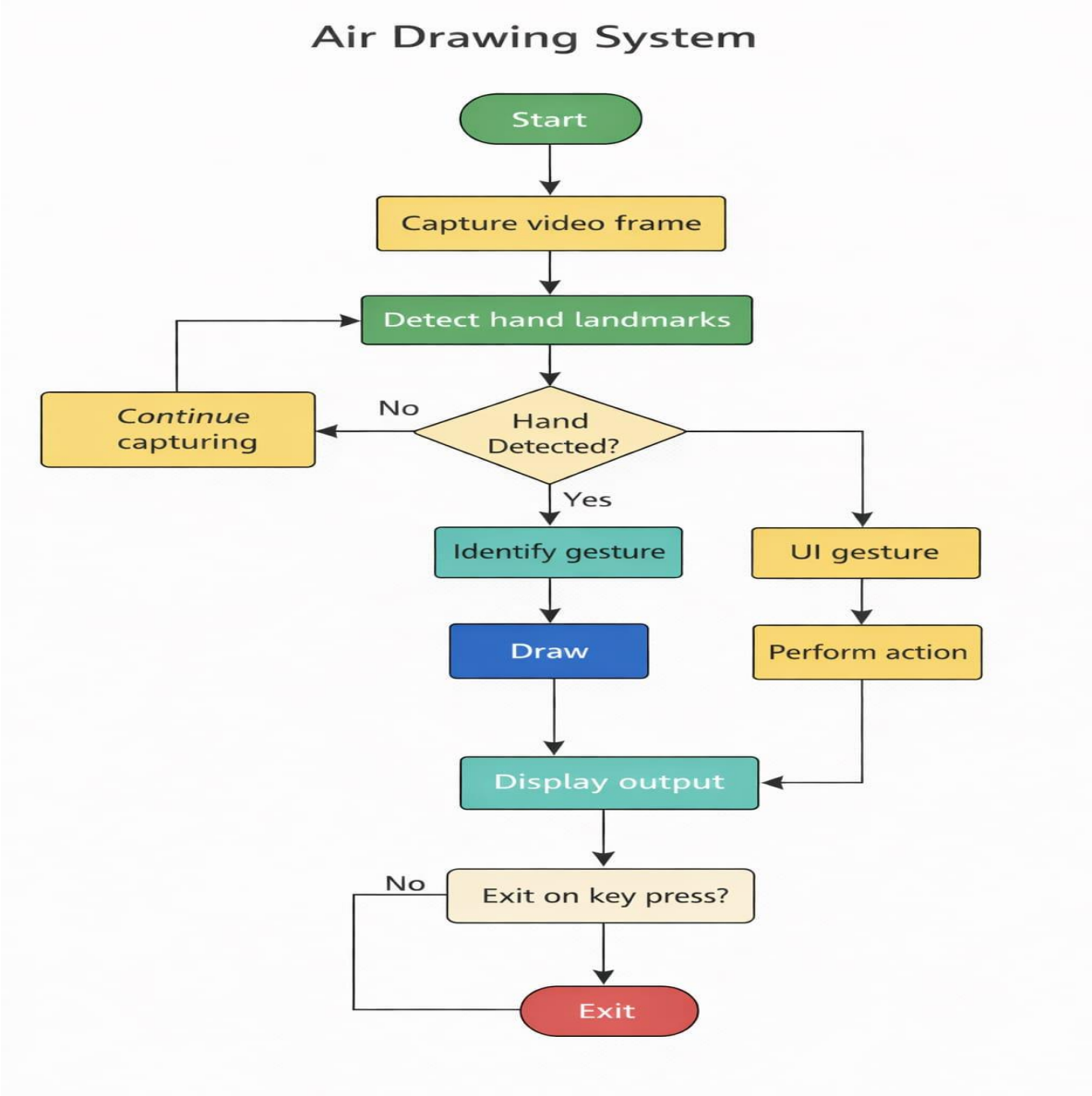


Fig 4.5 Activity Diagram

CHAPTER 5
IMPLEMENTATION

5.1 Algorithms

1. **Hand Landmark Detection (MediaPipe Hands)**
    - **Type:** Machine Learning / Computer Vision.
    - **Role in Project:** Detects and tracks hand landmarks in real time.
    - **Functionality:**
        - Captures video frames from webcam.
        - Uses MediaPipe to detect 21 hand landmarks.
        - Converts normalized coordinates into pixel values.
    - **Advantages in Project:**
        - High accuracy and fast performance.
        - Works without additional hardware.
    - **Outcome:** Accurate tracking of finger positions for gesture recognition.

2. **Gesture Recognition Algorithm**
    - **Type:** Rule-Based Gesture Detection
    - **Role in Project**: Identifies drawing and non-drawing gesture
    - **Functionality:**
        - Extracts index finger tip (landmark 8) and thumb tip (landmark 4)
        - Calculates vertical distance between thumb and index finger
        - If distance < threshold → stop drawing
        - Else → enable drawing
    - **Advantages in Project:**
        - Real-time gesture interpretation.
        - Simple and efficient logic.
    - **Outcome:** Enables intuitive control of drawing actions.

3. **Stroke Storage using Deque**
    - **Type:** Data Structure Algorithm
    - **Role in Project:** Stores drawing points efficiently
    - **Functionality:**
        - Uses deque to store points for each stroke.
        - Maintains separate lists for each color.
        - Creates new deque when drawing stops.
    - **Advantages in Project:**
        - Fast insertion and deletion.
        - Prevents unwanted line connections.
    - **Outcome:** Smooth and continuous drawing strokes.
4. **Drawing Algorithm (OpenCV Rendering)**
    - **Type:** Graphics Rendering
    - **Role in Project:** Displays drawing on screen.
    - **Functionality:**
        - Connects consecutive points using lines.
        - Draws on both live frame and virtual canvas.
        - Supports multiple colors.
    - **Advantages in Project**:
        - Real-time visualization
        - Smooth Rendering
        - **Outcome:** Visible drawing output on screen
5. **Color Selection and UI Interaction**
    - **Type:** Coordinate based selection
    - **Role in Project:** Allows user to select colors and clear canvas.
    - **Functionality:**
    - Defines button regions at top of screen
    - Detects finger position within button boundaries
    - Performs actions like color change or clearing canvas.
    - **Advantages in project:** Simple and intuitive interface
    - No need for physical controls
    - **Outcome:** Interactive virtual drawing environment

5.2 Architectural Components

1. Input Module

- Captures live video stream using a webcam.
- Converts captured frames into a suitable format for processing.
- Ensures continuous frame acquisition for real-time interaction.

2. Preprocessing Module

- Flips the frame horizontally to provide a mirror view.
- Converts BGR image to RGB format for MediaPipe processing.
- Enhances frame quality for better hand detection.

3. Hand Tracking Module Module

- Uses MediaPipe Hands model to detect hand landmarks.
- Identifies 21 key points of the hand.
- Extracts important landmarks such as index finger tip and thumb tip.

4. Gesture Recognition Module

- Analyses spatial relationship between landmarks.
- Determines drawing state based on thumb-index distance.
- **Differentiates between:**
  - Drawing Gesture
  - Stop Gesture
  - UI Interaction Gesture

5. Drawing Management Module

- Stores drawing points using deque data structures.
- Maintains separate storage for each color.
- Creates new stroke segments when drawing is interrupted.

6. User Interface Module

- Displays virtual buttons on the screen.
- **Handles:**

- Colour Selection
- Canvas Clearing

- Uses coordinate-based interaction for gesture control.

7. Rendering Module

- Draws lines between consecutive points.
- **Displays drawing on:**
  - Live video feed
  - Separate canvas window
- Ensures smooth and continuous rendering.

8. Output Module

- Displays final output using OpenCV windows.
- Provides real-time visual feedback to the user.

5.3 Feature Extraction

Step-1: Hand Landmark Detection

- Detects 21 hand landmarks using MediaPipe.
- The system uses MediaPipe Hands to detect hand landmarks in real time.
- MediaPipe identifies **21 key landmarks** on the hand, each representing joints and fingertips.
- These Landmarks include:
  - Wrist
  - Fingerjoints
  - Fingertips
- Each landmark provides normalized coordinates (x, y, z).

Step-2: Coordinate Conversion

- The detected landmarks are initially in normalized form (values between 0 and 1).
- These values are converted into pixel coordinates based on frame resolution.
- Converting to pixel coordinates allows mapping of hand positions directly onto the screen for drawing and interaction.

Step-3: Key Feature selection

- Among the 21 landmarks, only important points are selected:
- Index Finger Tip (Landmark 8) → used for drawing
- Thumb Tip (Landmark 4) → used for gesture control
- Reduces computational complexity and focuses only on relevant features for efficient processing.

Step-4: Distance-Based Feature Extraction

- The system calculates the distance between the thumb and index finger.
- This is used to determine whether drawing should be enabled or disabled.
- Ex: If distance < threshold → Stop drawing
- Ex: If distance ≥ threshold → Continue drawing
- This simple feature enables intuitive gesture-based control without complex models.

Step-5: Position-Based Feature Extraction

- The system checks the position of the index finger relative to predefined screen regions.
- These regions correspond to UI buttons such as:
  - Clear
  - Yellow
  - Blue
  - Green
  - Red
- Enables gesture-based interaction with the virtual interface.

Step-6: Motion tracking Features

- Continuous movement of the index finger is tracked across frames.
- The trajectory of movement is stored as a sequence of points.
- Allows smooth and natural drawing similar to real pen movement.

Step-7: Stroke Segmentation

- When drawing is paused (thumb close to index finger), a new stroke is created.
- Each stroke is stored separately using deque structures.
- Prevents unwanted connections between different drawing segments.

Step-8: Color-Based Feature Handling

- The system maintains separate feature sets for each color.
- Each color has its own list of stroke points.
- Enables multi-color drawing and better organization of drawing data.

Step-9: Temporal Feature Processing

- The system processes features frame-by-frame in real time.
- Maintains continuity of drawing across frames.
- Ensures smooth real-time performance without delay.

Step-10: Noise Handling and Stability

- Small unwanted movements are filtered by:
- Threshold Conditions
- Controlled point addition
- Prevents jitter in drawing

5.4 Packages /Libraries Used

1. **Computer Vision and Image Processing Libraries**
    - Role in Project**:** Core library for video processing and rendering
    - Captures live video stream from webcam.
    - Performs frame-by-frame image processing.
    - Draws shapes, lines, and UI elements on frames.
    - Displays output windows (live feed and canvas).

2. **Machine Learning and Hand Tracking Libraries**
    - Uses pre-trained deep learning models.
    - Uses pre-trained deep learning models
    - Provides accurate hand tracking even in dynamic conditions.
    - Mp.solution.heads- Hand tracking module.

3. **Numerical Computation libraries**
    - Creates and manipulates image matrices.
    - Used to initialize the drawing canvas.
    - Performs numerical computations.

4. **Data Structures and Utility Libraries**
    - Maintains ordered list of points.
    - Supports fast insertion and deletion.
    - Limits memory usage with fixed size.
    - Deque(maxlen=1024) - Store stroke points.
    - Appendleft() - Add new points.

5. **Python Standard Libraries**
    - Handles loops, conditions, and control statements.
    - Manages real-time execution of the system.
    - Integrates different modules smoothly.

5.5 Source Code

```
# All the imports go here
import cv2
import numpy as np
import mediapipe as mp
from collections import deque


# Giving different arrays to handle colour points of different colour
bpoints = [deque(maxlen=1024)]
gpoints = [deque(maxlen=1024)]
rpoints = [deque(maxlen=1024)]
ypoints = [deque(maxlen=1024)]


# These indexes will be used to mark the points in particular arrays of specific colour
blue_index = 0
green_index = 0
red_index = 0
yellow_index = 0

#The kernel to be used for dilation purpose
kernel = np.ones((5,5),np.uint8)

colors = [(255, 0, 0), (0, 255, 0), (0, 0, 255), (0, 255, 255)]
colorIndex = 0

# Here is code for Canvas setup
paintWindow = np.zeros((471,636,3)) + 255
paintWindow = cv2.rectangle(paintWindow, (40,1), (140,65), (0,0,0), 2)
paintWindow = cv2.rectangle(paintWindow, (160,1), (255,65), (255,0,0), 2)
paintWindow = cv2.rectangle(paintWindow, (275,1), (370,65), (0,255,0), 2)
paintWindow = cv2.rectangle(paintWindow, (390,1), (485,65), (0,0,255), 2)
paintWindow = cv2.rectangle(paintWindow, (505,1), (600,65), (0,255,255), 2)

cv2.putText(paintWindow, "CLEAR", (49, 33), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
(0, 0, 0), 2, cv2.LINE_AA)
cv2.putText(paintWindow, "BLUE", (185, 33), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
(0, 0, 0), 2, cv2.LINE_AA)
```

```
cv2.putText(paintWindow, "GREEN", (298, 33), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 2, cv2.LINE_AA)
cv2.putText(paintWindow, "RED", (420, 33), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 2, cv2.LINE_AA)
cv2.putText(paintWindow, "YELLOW", (520, 33), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 2, cv2.LINE_AA)
cv2.namedWindow('Paint', cv2.WINDOW_AUTOSIZE)


# initialize mediapipe
mpHands = mp.solutions.hands
hands = mpHands.Hands(max_num_hands=1, min_detection_confidence=0.7)
mpDraw = mp.solutions.drawing_utils


# Initialize the webcam
cap = cv2.VideoCapture(0)
ret = True
while ret:
    # Read each frame from the webcam
    ret, frame = cap.read()

    x, y, c = frame.shape

    # Flip the frame vertically
    frame = cv2.flip(frame, 1)
    #hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    framergb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    frame = cv2.rectangle(frame, (40,1), (140,65), (0,0,0), 2)
    frame = cv2.rectangle(frame, (160,1), (255,65), (255,0,0), 2)
    frame = cv2.rectangle(frame, (275,1), (370,65), (0,255,0), 2)
    frame = cv2.rectangle(frame, (390,1), (485,65), (0,0,255), 2)
    frame = cv2.rectangle(frame, (505,1), (600,65), (0,255,255), 2)
    cv2.putText(frame, "CLEAR", (49, 33), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 2, cv2.LINE_AA)
    cv2.putText(frame, "BLUE", (185, 33), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 2, cv2.LINE_AA)
    cv2.putText(frame, "GREEN", (298, 33), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 2, cv2.LINE_AA)
```

```
    cv2.putText(frame, "RED", (420, 33), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0),
2, cv2.LINE_AA)
    cv2.putText(frame, "YELLOW", (520, 33), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,
0, 0), 2, cv2.LINE_AA)
    #frame = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

    # Get hand landmark prediction
    result = hands.process(framergb)

    # post process the result
    if result.multi_hand_landmarks:
        landmarks = []
        for handslms in result.multi_hand_landmarks:
            for lm in handslms.landmark:
                # # print(id, lm)
                # print(lm.x)
                # print(lm.y)
                lmx = int(lm.x * 640)
                lmy = int(lm.y * 480)

                landmarks.append([lmx, lmy])


            # Drawing landmarks on frames
            mpDraw.draw_landmarks(frame, handslms, mpHands.HAND_CONNECTIONS)
        fore_finger = (landmarks[8][0],landmarks[8][1])
        center = fore_finger
        thumb = (landmarks[4][0],landmarks[4][1])
        cv2.circle(frame, center, 3, (0,255,0),-1)
        print(center[1]-thumb[1])
        if (thumb[1]-center[1]<30):
            bpoints.append(deque(maxlen=512))
            blue_index += 1
            gpoints.append(deque(maxlen=512))
            green_index += 1
            rpoints.append(deque(maxlen=512))
            red_index += 1
            ypoints.append(deque(maxlen=512))
            yellow_index += 1

        elif center[1] <= 65:
```

```
        if 40 <= center[0] <= 140: # Clear Button
            bpoints = [deque(maxlen=512)]
            gpoints = [deque(maxlen=512)]
            rpoints = [deque(maxlen=512)]
            ypoints = [deque(maxlen=512)]

            blue_index = 0
            green_index = 0
            red_index = 0
            yellow_index = 0

            paintWindow[67:,:,:] = 255
        elif 160 <= center[0] <= 255:
                colorIndex = 0 # Blue
        elif 275 <= center[0] <= 370:
                colorIndex = 1 # Green
        elif 390 <= center[0] <= 485:
                colorIndex = 2 # Red
        elif 505 <= center[0] <= 600:
                colorIndex = 3 # Yellow
    else :
        if colorIndex == 0:
            bpoints[blue_index].appendleft(center)
        elif colorIndex == 1:
            gpoints[green_index].appendleft(center)
        elif colorIndex == 2:
            rpoints[red_index].appendleft(center)
        elif colorIndex == 3:
            ypoints[yellow_index].appendleft(center)
# Append the next deques when nothing is detected to avois messing up
else:
    bpoints.append(deque(maxlen=512))
    blue_index += 1
    gpoints.append(deque(maxlen=512))
    green_index += 1
    rpoints.append(deque(maxlen=512))
    red_index += 1
    ypoints.append(deque(maxlen=512))
    yellow_index += 1

# Draw lines of all the colors on the canvas and frame
```

```
    points = [bpoints, gpoints, rpoints, ypoints]
    # for j in range(len(points[0])):
    #         for k in range(1, len(points[0][j])):
    #             if points[0][j][k - 1] is None or points[0][j][k] is None:
    #                 continue
    #             cv2.line(paintWindow, points[0][j][k - 1], points[0][j][k], colors[0], 2)
    for i in range(len(points)):
        for j in range(len(points[i])):
            for k in range(1, len(points[i][j])):
                if points[i][j][k - 1] is None or points[i][j][k] is None:
                    continue
                cv2.line(frame, points[i][j][k - 1], points[i][j][k], colors[i], 2)
                cv2.line(paintWindow, points[i][j][k - 1], points[i][j][k], colors[i], 2)

    cv2.imshow("Output", frame)
    cv2.imshow("Paint", paintWindow)

    if cv2.waitKey(1) == ord('q'):
        break

# release the webcam and destroy all active windows
cap.release()
cv2.destroyAllWindows()
```

5.6 Output Screens

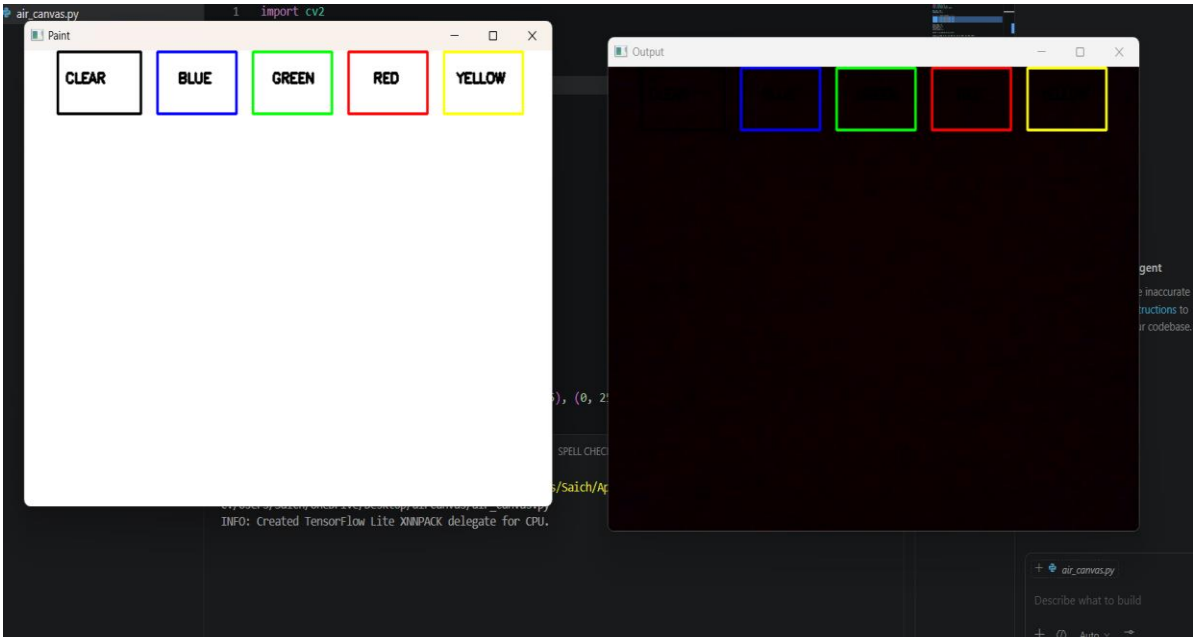


Fig: Paint and video interface

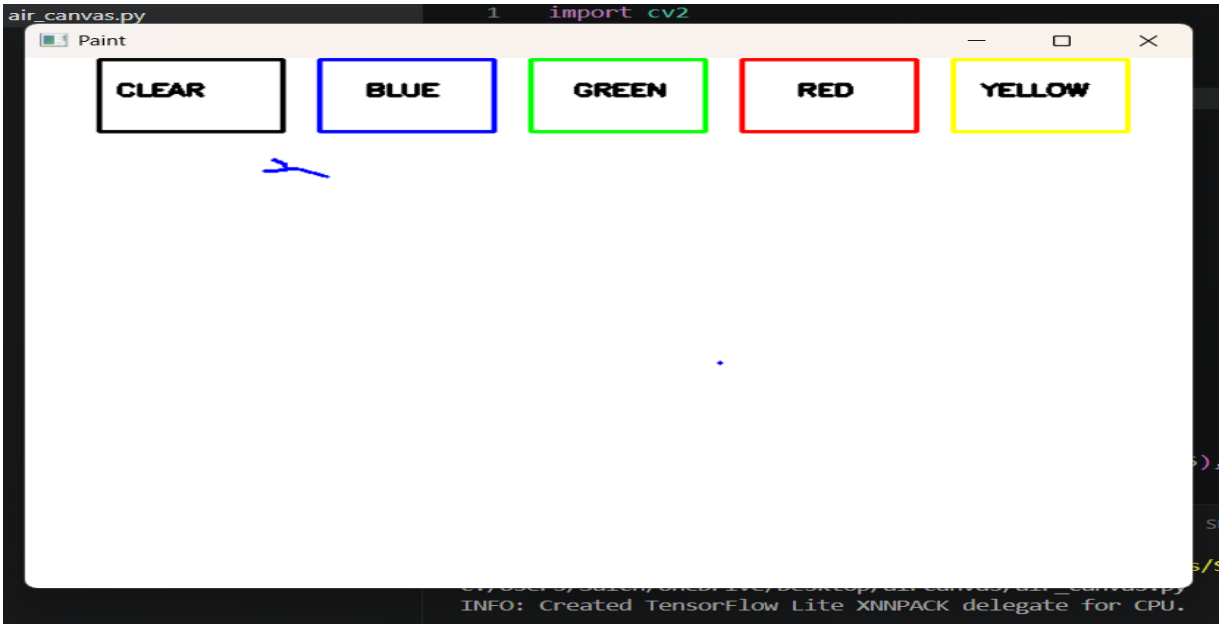


Fig: Canvas paint window

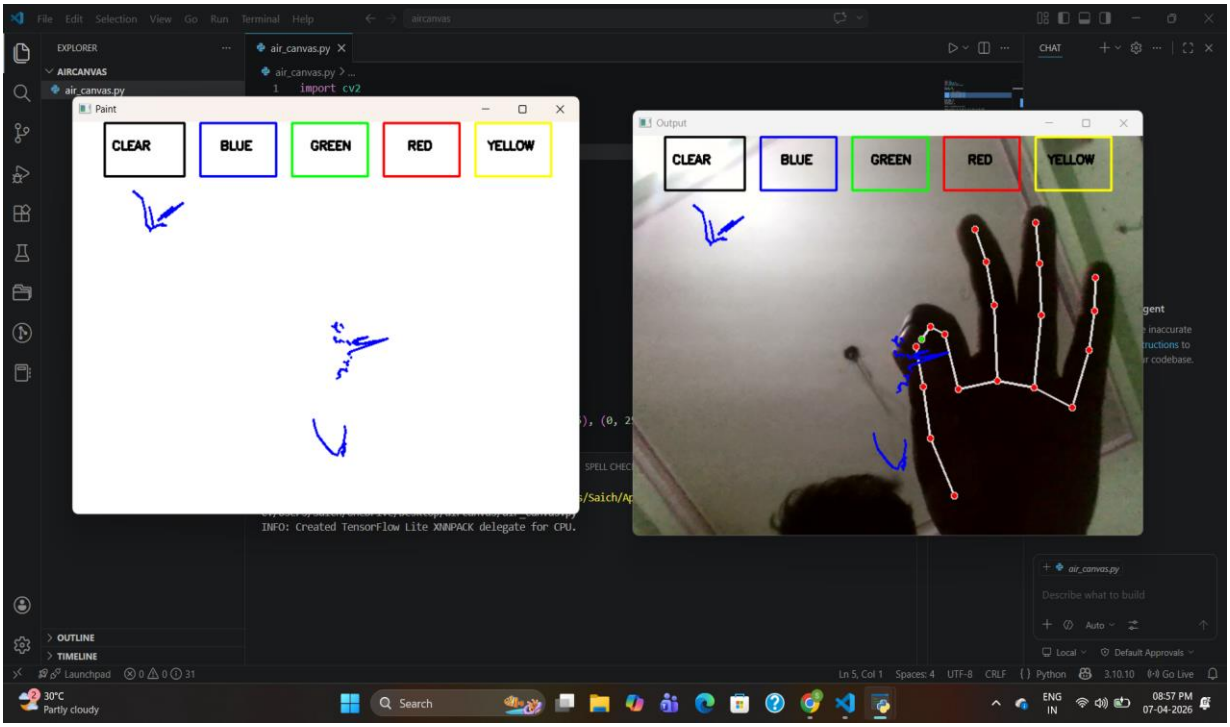


Fig: Hand gesture recognition using MediaPipe

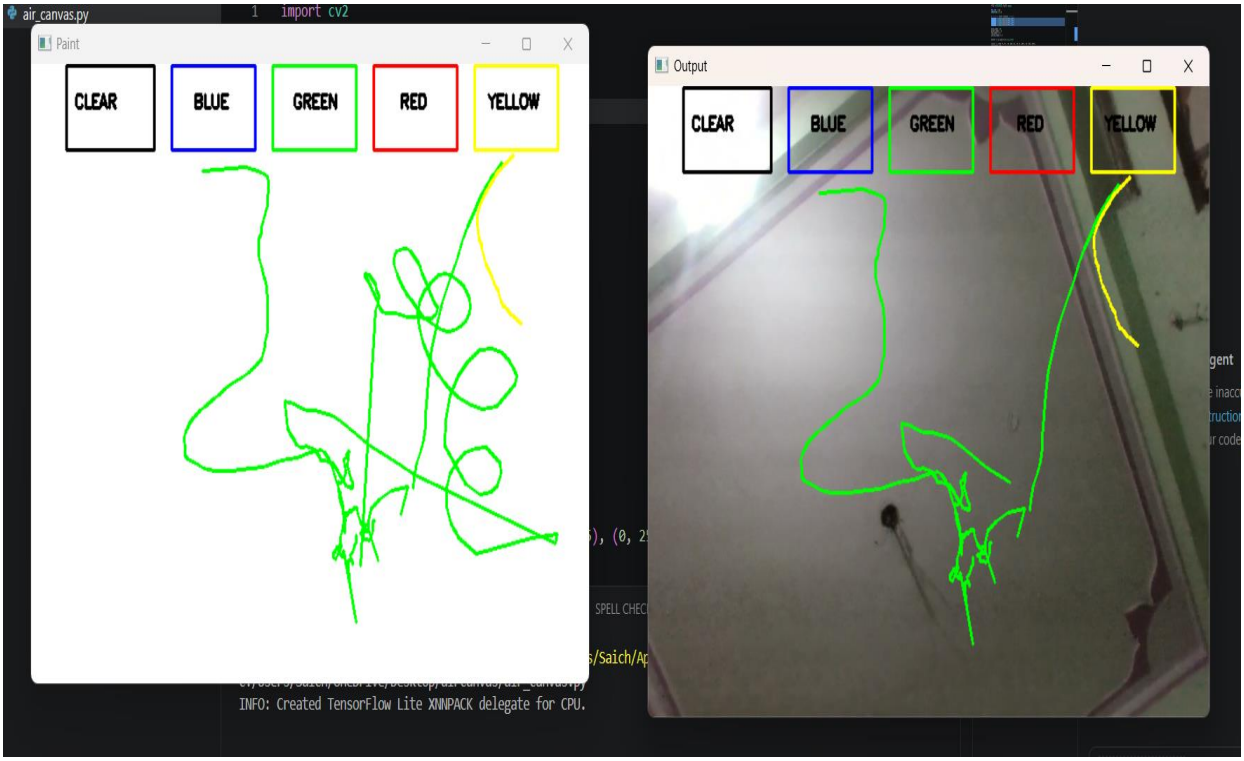


Fig: Showing how canvas drawn

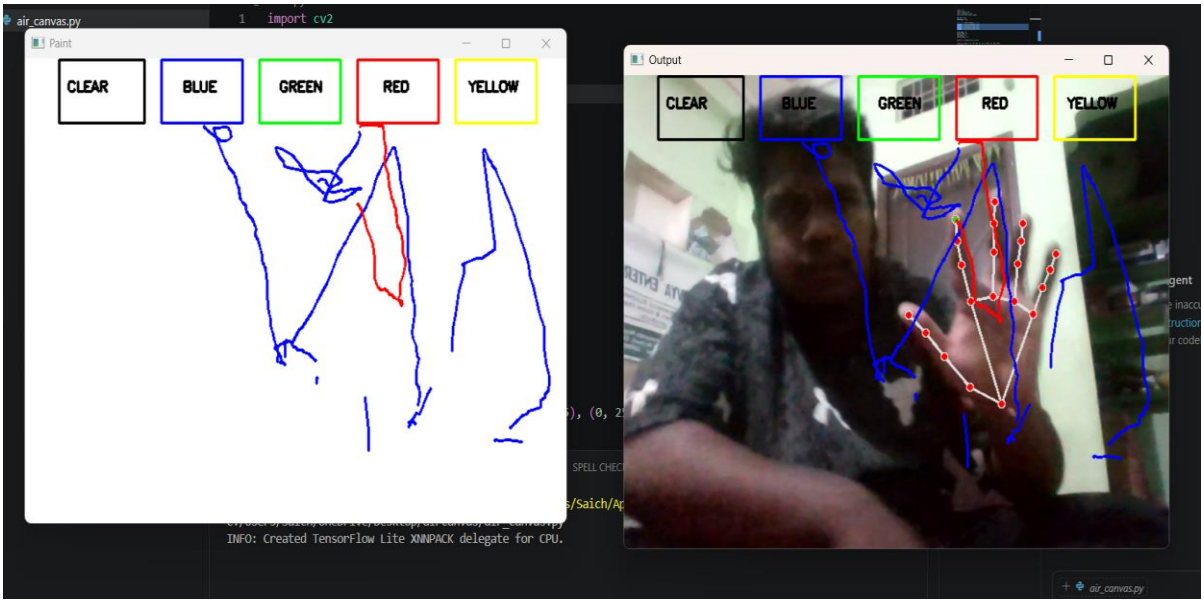


Fig: Writing with free hand

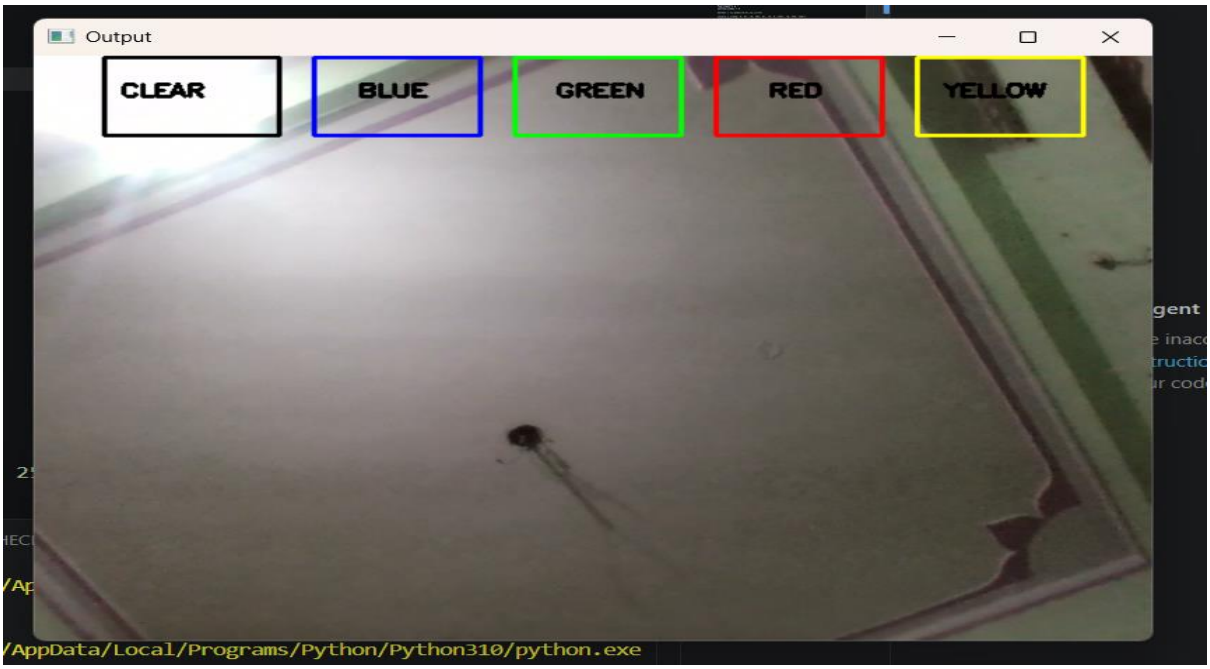


Fig: video capture window

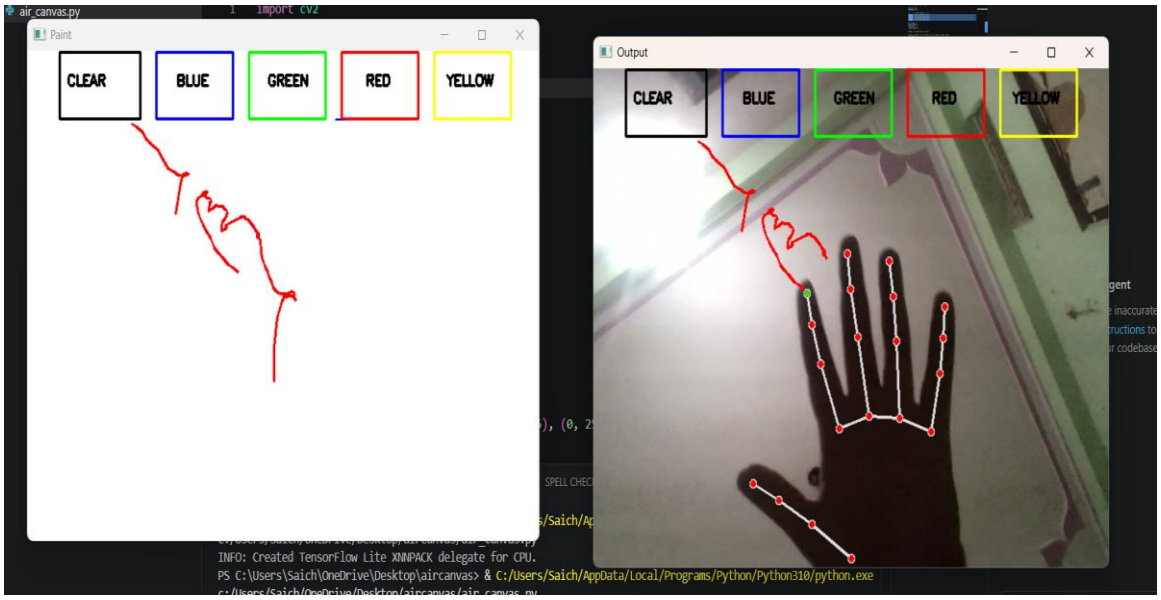


Fig: Hand Recognition

CHAPTER 6

SYSTEM TESTING

6.1 Test Cases

1. Video Input Test Cases

- Check if webcam initializes correctly.
- Verify continuous frame capture without delay.
- Test system behavior when camera is unavailable.
- Validate frame resolution and clarity.

2. Validate frame resolution and clarity

- Verify detection of hand in different positions.
- Check detection accuracy under different lighting conditions.
- Test system when no hand is present.
- Ensure correct identification of all 21 landmarks.
- Validate detection speed for real-time performance.

3. Gesture Recognition Test Cases

- Verify correct detection of drawing gesture (index finger up).
- Check detection of stop gesture (thumb close to index finger).
- Test threshold values for gesture accuracy.
- Ensure no false gesture detection.
- Validate performance with fast hand movements.

4. Drawing Functionality Test Cases

- Verify drawing starts and stops correctly.

- Check smoothness of drawing lines.
- Ensure no unwanted connections between stroke.
- Detect overfitting using training vs validation accuracy comparison.
- Validate multi-color drawing functionality.
- Test continuous drawing over long duration.

5. User Interface Test Cases

- Verify color selection using finger position.
- Check CLEAR button functionality.
- Ensure UI elements are correctly displayed.
- Validate interaction accuracy with buttons.

6. System Performance Test Cases

- Measure frame processing speed (FPS).
- Check system response time.
- Test performance on different hardware configurations.
- Ensure no lag during real-time drawing.

7. Error Handling Test Cases

- Handle absence of hand detection
- Handle sudden movement or occlusion
- Validate system stability during continuous execution
- Check proper exit on key press.

6.2 Results and Discussions

Results:

- The system was successfully implemented using OpenCV and MediaPipe.

- Real-time hand tracking was achieved with high accuracy.
- Gesture-based drawing worked smoothly with minimal delay.
- Multi-color drawing and canvas clearing features functioned correctly.

Performance observation:

- The system maintained real-time performance with smooth frame rates.
- Hand detection was accurate under normal lighting conditions.
- Gesture recognition worked effectively with minimal errors.

Discussion:

Accuracy of Hand tracking:

- MediaPipe provided reliable detection of hand landmarks.
- Minor inaccuracies occurred under poor lighting conditions.

Gesture recognition Efficiency:

- The simple rule-based approach worked efficiently.
- Threshold tuning improved gesture detection accuracy.

Drawing Quality:

- Drawing was smooth and continuous due to deque data structure.
- Stroke segmentation prevented unwanted connections.

System Limitations:

- Performance may decrease in low lighting.
- Requires stable camera positioning
- Limited to single-hand detection.

Conclusion from Results:

- Best performance was observed under good lightning conditions**.**
- Slight inaccuracies were observed in low-light environments.
- Fast hand movements sometimes caused minor tracking delays.
- Gesture detection was reliable for normal hand movements.

6.2.1 Datasets

Dataset Used:

- No pre-defined dataset is used.
- The system uses real-time webcam input.

Input Data:

- Live video frames.
- Hand gestures captured dynamically.

Handling Noise and Errors:

- Small movements filtered using threshold conditions.
- Stable gesture detection ensured through continuous tracking.

6.3 Performance Evaluation

Performance Evaluation

The performance of the system was evaluated based on the following parameters:

1. Accuracy:

- Hand detection accuracy was high under normal conditions.
- Gesture recognition worked correctly in most cases.

2. Response Time:

- The system processed frames in real time with minimal delay.
- Drawing response was nearly instantaneous.

3. Stability:

- The application ran continuously without crashes.
- Stable tracking was maintained during moderate movement.

4. Usability:

- User interface was simple and intuitive.
- Easy color selection and drawing experience.

5. Efficiency:

- Efficient memory usage using deque data structures.

- Low computational overhead suitable for standard systems.
- **Response Time:**Real-time response with minimal delay
- **Frame Rate (FPS):**Maintains smooth performance (20–30 FPS depending on system)
- **Drawing Smoothness:**Continuous and stable stroke rendering

CHAPTER 7 CONCLUSION & FUTURE ENHANCEMENTS

Conclusion:

The Real-Time Air Drawing Application demonstrates an effective implementation of gesture-based human-computer interaction using computer vision and machine learning techniques. By leveraging MediaPipe for hand landmark detection and OpenCV for real-time image processing, the system enables users to draw in the air without any physical contact or additional hardware. The application successfully interprets hand gestures, allowing users to control drawing actions such as enabling or disabling drawing, selecting colors, and clearing the canvas.

The system achieves real-time performance with smooth and continuous stroke rendering, making the interaction intuitive and user-friendly. The use of efficient data structures like deque ensures proper handling of drawing strokes and prevents unwanted connections. Overall, the project highlights the potential of touchless interfaces in enhancing user experience and provides a practical solution for applications in virtual whiteboards, education, and creative design tools. This work demonstrates how computer vision can be used to bridge the gap between human gestures and digital systems effectively.

Future Enhancements:

While the current system performs efficiently, several improvements can be made to enhance its functionality and usability:

- **Multi Hand Detection:** Extend the system to support multiple hands simultaneously. Enable collaborative drawing or advanced gesture controls
- **Advance gesture recognition**: Implement more complex gestures using machine learning models. Adding features like Eraser Tool, Shape drawing, Zoom and scaling.

- **Integration with AI Models**: Use deep learning models for more accurate gesture classification. Improve robustness under different lighting and background conditions.
- **Mobile and Web development**: Convert the system into a mobile or web-based application. Enable wider accessibility and usability.
- **Save and Export Functionality**: Allow users to save their drawings as images or PDFs. Enable sharing options.
- **Performance Optimization**: Improve processing speed and reduce latency. Optimize for low-end devices.
- **Augmented Reality Integration**: Integrate with AR technologies to enhance user experience. Enable 3D drawing in space.
- **Voice Control Integration:** Combine gesture recognition with voice commands. Provide a more interactive and hands-free experience.
- **AI Based Drawing Assistance:** Suggest shapes or auto-complete drawings using AI models. Provide handwriting recognition features.
- **Virtual Whiteboard Integration:** Use the system for online teaching and presentations. Integrate with platforms like Zoom or Google Meet.
- **Shape Recognition:** Integrate machine learning models to detect shapes (circle, square, line). Automatically correct rough sketches into proper geometric shapes.

CHAPTER 8
REFERENCES

[1] F. Chollet, *Deep Learning with Python*, Manning Publications, 2018.

[2] R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2010.

[3] OpenCV Documentation, "Open Source Computer Vision Library," Available: https://opencv.org/.

[4] MediaPipe Documentation, "Google MediaPipe Hands Solution," Available: https://developers.google.com/mediapipe.

[5] G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*, O'Reilly Media, 2008.

[6] NumPy Documentation, "NumPy Library for Numerical Computing," Available: https://numpy.org/.

[7] Python Software Foundation, "Python Documentation," Available: https://docs.python.org/3/.

[8] Research Papers on Hand Gesture Recognition and Human-Computer Interaction, IEEE Xplore Digital Library.

[9] S. Mitra and T. Acharya, "Gesture Recognition: A Survey," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 37, no. 3, pp. 311–324, 2007.